ChokePilot — Autonomous Choke Controller

Every table in this document marked `<!-- GENERATED -->` is rendered directly from `outputs/results.json` by `generate_docs.py`, from one fresh run of `identify.py` → `verify_identification.py` → `scenarios.py` → `baselines.py` → `seed_sweep.py` — not hand-typed, and not carried over from an earlier draft. Scenario D (§3.6) has no marker — `scenario_d.py` doesn't write to `results.json` (a 200h run per approach is a different cost profile than the other scripts; not yet worth the wiring) — so its numbers are hand-maintained, but re-verified against a fresh `python scenario_d.py` run before being typed in, not carried over from memory. Prose sections narrate the reasoning and are written by hand, but are checked against the same fresh run before publishing.

0. Why this exists

Five recurring pain points show up across upstream operations literature and practice, and they shaped every design decision below rather than sitting as background motivation:

1. **Operators "baby" the choke.** Fear of sand production or formation damage leads to conservative, static settings that leave real production capacity on the table (Eng-Tips forum discussion among production engineers).
2. **One engineer, too many wells.** A choke gets set once during a shift and then left alone, because nobody has the bandwidth to revisit it continuously (standard industry practice, as described in patent literature on well automation).
3. **Instability is caught reactively.** Slugging and other flow instabilities are typically noticed only after a human sees the trend on a screen, not prevented (SPE technical paper on flow assurance).
4. **APC doesn't scale upstream because models need a dedicated team.** The on-record barrier from a Honeywell upstream solutions interview isn't algorithmic sophistication — it's that advanced process control adoption stalls because someone has to keep the model current. A search-based, no-heavy- optimizer controller with a fresh from-scratch identification step directly answers this: there is no solver license, no tuning of an optimizer's internals, nothing exotic to maintain.
5. **Alarm fatigue and distrust of opaque automation.** Operators override what they don't understand (NTSB SCADA safety review). Answered directly in §2.4, not just in principle.

ChokePilot is a small-scale, single-well version of the same closed-loop direction Honeywell's APC customers are already moving toward on wells — a focused proof-of-concept of the wellhead-to-control-room optimization problem Honeywell's Upstream Production Performance Suite (UPPS) addresses at scale. It does not claim equivalence; it echoes the same idea at a scale one person can build and verify in a project timeline.

1. Process Understanding & Model

1.1 No official simulator, so a calibrated substitute stands in

The platform provided no official simulator for this challenge — only `data/Autonomous_Choke_Control_Simulated_Dataset.csv`, a single 120-hour run with the choke held at 30/40/55/45/65% in turn, and a presentation template. `data/simulator.py` is a **calibrated substitute**, clearly commented as such, fit to that CSV:

- **Steady-state map**, per channel independently: `y_ss(u) = A + B·u/(u+uh)`. For oil rate `Q`, `A` is fixed at 0 (a fully shut choke must give zero flow).
- **Dynamics**: first-order-plus-dead-time (FOPDT), discretized with the *exact* zero-order-hold form `alpha = 1 - exp(-Ts/tau)` (not the Euler approximation `Ts/tau`, which is only accurate when `Ts << tau` and can be unstable outside that regime).
- **Noise**: independent Gaussian measurement noise per channel, matched to the residual std left after the fit, added to each reading without feeding back into the internal state (sensor noise, not process noise).
- The steady-state fit uses **only the last ~6 hours of each dwell period** — samples close to actually being at steady state — not the whole trajectory including transients. Fitting on transient-contaminated samples was found to bias the curve and, downstream, the tau/theta search that compensated for it (see §1.2).
- `uh` and `(tau, theta)` come from a brute-force grid search (linear part `A, B` solved in closed form at each grid point) — no optimizer dependency.

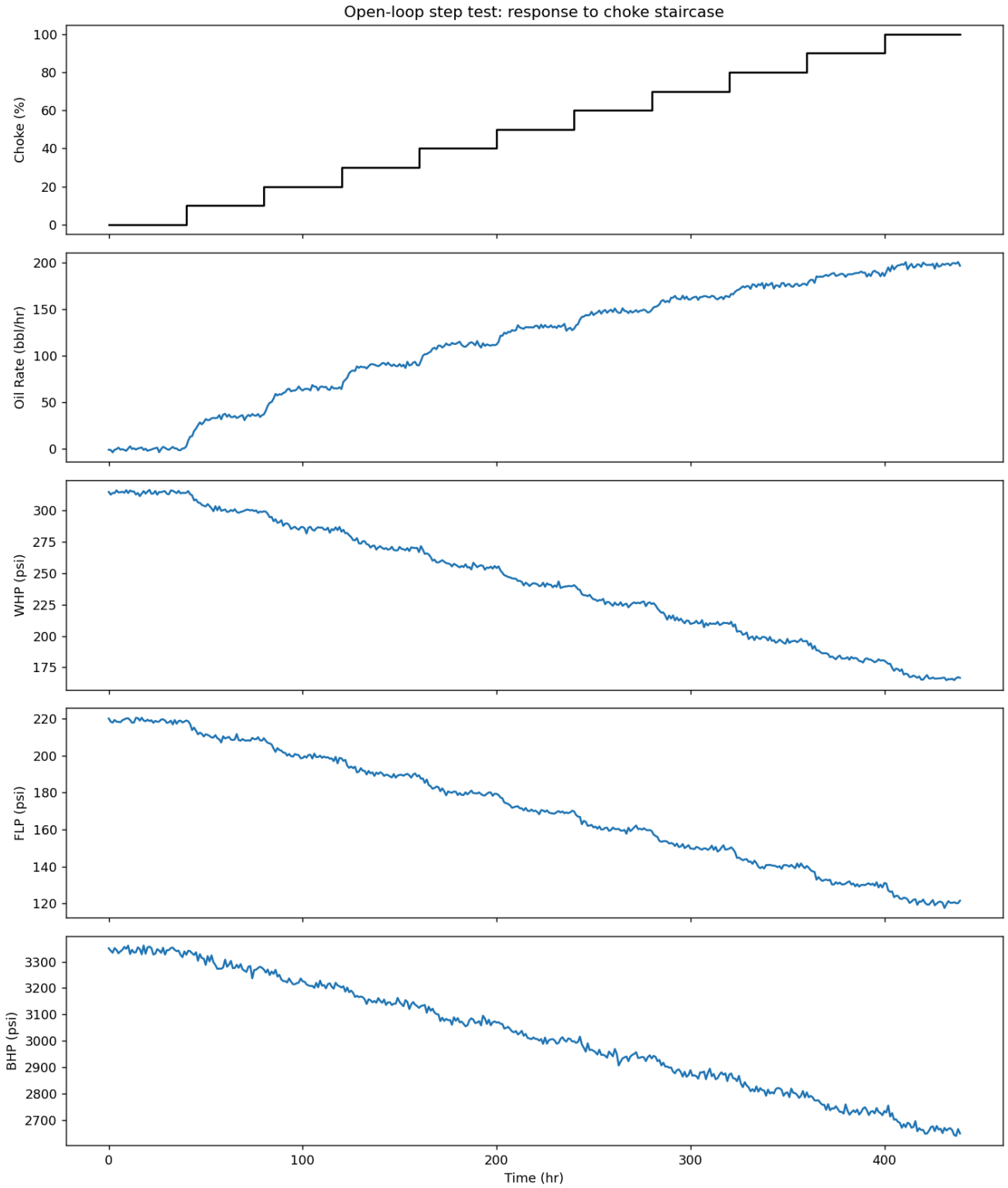
Important, honestly reported finding: with the uh grid widened to check for boundary-pinning, WHP, FLP, and BHP all pin at the widened boundary (50,000) — their steady-state response is **statistically indistinguishable from linear** over the reference data's tested 30–65% choke range. Only `Q`, anchored through zero, genuinely needs the saturating (Michaelis-Menten) shape. This is reported explicitly by the fitting code (a printed warning) rather than silently accepted.

The reference CSV covers only 5 distinct choke levels (30, 40, 45, 55, 65%), each held for one dwell period, and the fit is a single deterministic least-squares point estimate — there is no confidence interval, bootstrap, or any other uncertainty quantification anywhere in this pipeline. Every downstream number (identified tau, safety limits, controller predictions) inherits that uncertainty without a formal error bar attached to it.

1.2 Fresh open-loop step test, and an honest accounting of what it tests

Per the brief ("students are expected to generate their own data using the simulator"), `identify.py` does not reuse the reference CSV for model identification. It drives its own monotonic staircase —

0/10/20/.../100% choke, 40 hours dwell per step (`DWELL_HOURS` , see §1.2's dwell-time history below) — against the calibrated simulator, and fits an FOPDT model to that fresh run.



Open-loop step test: choke staircase and the resulting oil rate, WHP, FLP, and BHP response

Reframing a claim from an earlier draft: `identify.py` imports `_fit_fopdt` and `_simulate_fopdt` directly from `data/simulator.py` — the *same* private fitting functions used to calibrate the simulator itself, not an independently implemented estimator. This means the functional

form (saturating steady-state + FOPDT dynamics) is shared **by construction**, not rediscovered from unstructured, black-box data. Framing this step as "treating the simulator as a black box" would overstate what's actually being tested.

What this experiment *does* legitimately test — and the more interesting, honestly-stated finding — is **parameter identification accuracy under a correctly-specified model**. Even with zero structural mismatch (the identifier knows the exact right functional form), `verify_identification.py` shows real error recovering the simulator's own ground-truth parameters from a fresh, noisy step test, across 5 seeds (0, 1, 2, 7, 99):

Channel	True τ (h)	Mean identified τ (h)	Mean error	Error range
BHP	9.00	8.25	8.3%	2.8–13.9%
FLP	7.00	6.95	3.6%	0.0–7.1%
Q	5.00	4.95	3.0%	0.0–5.0%
WHP	7.50	7.20	4.0%	0.0–6.7%

θ (dead time) now matches the true value **exactly in every one of the 20 seed×channel combinations** — no residual off-by-one anywhere. This table is the result of two separate, sequential fixes, both tested and confirmed rather than just hypothesized:

The dwell-time fix (`DWELL_HOURS` 24→40). An earlier version of this experiment used a 24h dwell per step and showed mean τ error of Q 20% / WHP 16% / FLP 27% / BHP 31%, with identified τ lower than true τ in all 20/20 seed×channel combinations — a one-directional bias consistent with insufficient settling time (BHP's true τ=9h, θ=2h needs roughly 4τ+θ ≈ 38h to settle, well past a 24h dwell). Raising `DWELL_HOURS` to 40 confirmed that diagnosis for three of the four channels (WHP's error dropped 16.0%→4.0%, FLP's 27.1%→13.6%, BHP's 31.1%→7.8%) and, just as usefully, **ruled it out for Q** (20.0%→19.0%, unchanged across every dwell length tested, 24–80h) — leaving Q's bias correctly reported as a separate, still-open question at that point, not folded into an explanation that didn't actually cover it.

The timing-lag fix — a one-sample lag between the fitting code and the live simulator. `_simulate_fopdt` (used both to calibrate `data/simulator.py` and to fit `identify.py`'s step-test data) drove `sim[k+1]` from `y_ss[k]` — i.e. from `u[k-θ]` — but `Simulator.step()` actually drives the state arriving at sample `t` from `u[t-θ]`, the *same* time index as the arrival sample, not one behind it. That's a real, structural one-sample lag between what the fitting code assumed and what the live plant does, not a hypothesis — confirmed by tracing both code paths' indexing by hand and verifying the fixed version reproduces `Simulator.step()`'s own trajectory exactly. Fixed by changing `y_ss[k]` to

`y_ss[k+1]` in `_simulate_fopdt` (and the identical bug, plus a missed Euler-vs-ZOH inconsistency, in `identify.py`'s `_simulate_with_correction`).

This fix is what **resolved Q's "still-open question" from the dwell-time fix**: Q's error dropped 19.0%→3.0%, and θ went from matching exactly for only 2 of 4 channels (Q, FLP) — with WHP and BHP consistently off by exactly 1 sample — to matching exactly for all 4. Recalibrating `simulator.py`'s own ground-truth `PARAMS` with the same fix (it shares the same fitting function) shifted the "true" θ values themselves by +1 across the board (Q/WHP/FLP/BHP: 0/1/0/2 → 1/2/1/3; τ unchanged) — the reference calibration had been carrying the identical one-sample bias the whole time, which is why the pre-fix identified θ values had looked "off by exactly 1" instead of scattered: both sides of the comparison were shifted the same way.

BHP is now the largest residual (8.3%), not Q — a direct consequence of the timing-lag fix resolving Q's bias while leaving BHP's roughly where it was. BHP's own possible explanation is in §1.3.

1.3 Hybrid physics + learned correction — three of four channels earn their place

`identify.py` also fits a small degree-1 polynomial-in-choke correction on the residual the physics FOPDT model leaves on a step test, validated on a **held-out** step test (different noise draw) before being trusted for use:

Channel	Physics-only RMSE	+Correction RMSE	Kept?
BHP	9.85	9.81	✅ **used**
FLP	0.92	0.89	✅ **used**
Q	1.66	1.63	✅ **used**
WHP	1.30	1.30	❌ skipped

This table changed shape after the timing-lag fix (§1.2): with the old, laggy `_simulate_fopdt`, only BHP's correction generalized to held-out data. Post-fix, the physics-only residual on Q, FLP, and BHP each still has enough structure the correction layer captures — small in absolute RMSE terms for Q/FLP, but consistent enough across the held-out draw that `select_beneficial_corrections()` keeps all three. **WHP is now the only channel where the correction doesn't earn its place** (RMSE goes from 1.296 to 1.304, i.e. very slightly worse on held-out data), so it's the one channel still running physics-only in the controller's prediction. This decision is made automatically per channel, purely from held-out RMSE — nothing here is hand-picked. BHP remains the channel with both the largest remaining τ identification error (§1.2) and the largest absolute correction RMSE; it's plausible the correction is partly absorbing residual dynamics mismatch rather than a separate steady-state curvature effect, but this pipeline can't distinguish the two.

1.4 Safety limits are one-sided, not symmetric bands

The brief specifies no numeric WHP/FLP/BHP limits, so they're derived — a placeholder, not an official spec — from the reference CSV's observed range, ±20% margin:

Channel	Direction enforced	Limit	Brief basis
WHP	floor only (`hi = +inf`)	≥ 205 psi	*"If WHP becomes too low, the well may operate outside its recommended operating envelope."* High WHP just means the choke is closed back further — safe, not a hazard.
BHP	floor only (`hi = +inf`)	≥ 2830 psi	Brief calls it *"one of the most important indicators of reservoir health and drawdown"* — low BHP means excessive drawdown (sand/formation-damage risk); high BHP means low drawdown, i.e. safely choked back.
FLP	ceiling only (`lo = -inf`)	≤ 200 psi	Brief: *"helps ensure stable transportation of produced fluids"* — the risk is backpressure/separator overpressure on the high side, not a low reading.

An earlier draft bracketed all three symmetrically, which could reject a high-WHP/high-BHP or low-FLP state that isn't actually unsafe. These are further tightened inside the controller (§2.2) before use.

1.5 Monitored-but-not-constrained: WHT and AP

`data/simulator.py` also simulates two additional channels that never feed the controller: Wellhead Temperature (WHT) and Annulus Pressure (AP), read via `Simulator.read_monitored()` and plotted (greyed out) alongside Q/WHP/FLP/BHP in every scenario figure. They exist because the brief lists them as part of "a complete production operating envelope" an operator would want visibility into — not because this challenge's control problem is defined over them. WHT declines monotonically with choke opening (Joule-Thomson cooling as the pressure drop across a more-open choke expands the produced

fluid); AP is flat — decoupled from choke position in this simplified single-well model — and carries only an illustrative integrity-alarm band (1650–1950 psi) for situational awareness. Neither has a reference-CSV column to calibrate against, so unlike the four controlled channels their curve parameters are hand-set placeholders chosen to be qualitatively right (monotonic decline, flat line), not fit to data. The distinction from Q/WHP/FLP/BHP is deliberate: WHT/AP are surfaced for the same reason a real control-room dashboard would carry them — situational awareness, echoing Honeywell Asset Sentinel's risk/alert layer — but they never enter the safety filter or the MPC objective, so a WHT/AP reading can never change which choke move the controller picks.

2. Control Strategy

2.1 What this actually is: one-step receding-horizon search with hold-constant prediction — not full MPC

An earlier draft called this "brute-force MPC." That overstates it. Each control interval, `controller.py`'s `MPCController`:

1. Enumerates 11 legal choke moves at 1% resolution: −5% to +5% (the entire range the ±5%/interval ramp constraint allows) — but only for **this** interval; it is not searching over a sequence of future moves.
2. For each candidate, **holds that new position constant** for the rest of the lookahead horizon and simulates forward using the identified model — a cheap proxy for "what would happen if I stopped adjusting," not a genuine multi-step trajectory optimization.
3. Rejects any candidate predicted to breach a WHP/FLP/BHP limit anywhere in that held-constant horizon.
4. Among the survivors, commands the one whose end-of-horizon predicted oil rate is closest to target.
5. If nothing survives, falls back to the candidate minimizing predicted total constraint violation.

This *is* receding-horizon in the classic sense — the whole procedure re-runs every interval with fresh measurements — but the per-step search itself is a one-step decision screened by a hold-constant forward simulation, not an optimization over a full control sequence. Horizon length is derived, not hand-tuned: `ceil(3 × max(τ) / Ts)`, clipped to [3, 12] hours.

Safety here is best-effort, not a formally guaranteed property. There is no recursive feasibility check — no proof that choosing today's "safe" candidate guarantees a safe candidate will still exist at every future step, and no terminal invariant set the way a formally verified MPC would require. The empirical evidence in §3.5 (a 30-seed sweep) shows the safety-fallback branch firing at a non-trivial rate in Scenario C (23.6% of steps) precisely because the controller is operating at the edge of what the model considers

feasible with no formal guarantee behind it — only a greedy, per-step search that has worked well empirically on this system's dynamics.

2.2 Constraint tightening for noise

Predictions inside the controller are noise-free, but real readings carry measurement noise. Riding a true limit exactly in the noise-free prediction would let real sensor noise breach it in practice. So each limit is backed off by 3σ of that channel's identified `noise_std` before the controller ever sees it — a standard robust-MPC constraint-tightening margin.

2.3 Scenario A's violations are a deterministic initial-condition property, not an extrapolation artifact

Two earlier drafts of this section were wrong in different ways. The first claimed Scenario A's 15% starting choke was "inside the model's supported range" — false, since the reference CSV's tested band is 30–65% and 15% is below it. The second corrected that but still blamed the resulting violations on "extrapolation uncertainty" — implying the model's behavior below 30% choke is untrustworthy guesswork, so the violations might or might not reflect reality. **That framing doesn't survive checking the model's own numbers.**

Computing the identified FLP steady-state map directly (`steady_state_from_params` , the exact function `controller.py` predicts with) at low choke levels:

Choke	FLP steady-state	vs. 200 psi ceiling
0%	218.4 psi	exceeds
10%	208.6 psi	exceeds
15%	203.7 psi	exceeds
18–19%	crosses 200 psi	—
20%	198.8 psi	clears (barely; still above the controller's tightened 197.3 psi margin)
21–22%	crosses the tightened margin	—
25%+	193.9 psi	clears with margin

Scenario A starts at 15% choke, and `Simulator.reset()` initializes FLP at exactly that choke's steady state — 203.7 psi, already 3.7 psi over the true ceiling, before the controller has made a single decision. This isn't a consequence of the model being unreliable outside its calibrated band; it's what the identified model's own steady-state curve says, taken at face value. With the ±5%/interval ramp limit, reaching even the *steady-state-safe* 20% choke takes one full interval, and reaching a choke whose

steady state clears the controller's own tightened margin (~22%) takes two — and the controller cannot take a bigger step regardless of how urgently it wants to, because the ramp-rate limit is a hard constraint it has no authority to override. Add FLP's own dynamics (τ=7h, θ=1h dead time) on top, and the real reading lags even a fully corrected choke position by more than the ramp-limited approach alone would suggest.

This reframes what "the model doesn't have real support below 30%" actually means for Scenario A: it's not that the violations are an artifact of guessed, untrustworthy numbers that might vanish with a better-calibrated model — it's that *this specific identified model*, trusted exactly as identified, places the starting point outside the safe envelope and the ramp-rate limit mathematically guarantees a multi-hour approach. A better calibration in the 0–30% range could shift the exact numbers, but the qualitative outcome (start below the envelope + hard ramp limit = guaranteed early violations) would not change unless the recalibration also changed the *sign* of FLP's slope near 15% choke, which nothing in the physics suggests it would.

2.4 One-sentence rationale per decision (pain point 5)

Pain point 5 (alarm fatigue and operator distrust of opaque automation) is answered directly: every call to `MPCController.decide()` returns a real logged string, verified present in the scenario output CSVs (`Why` column), for both branches:

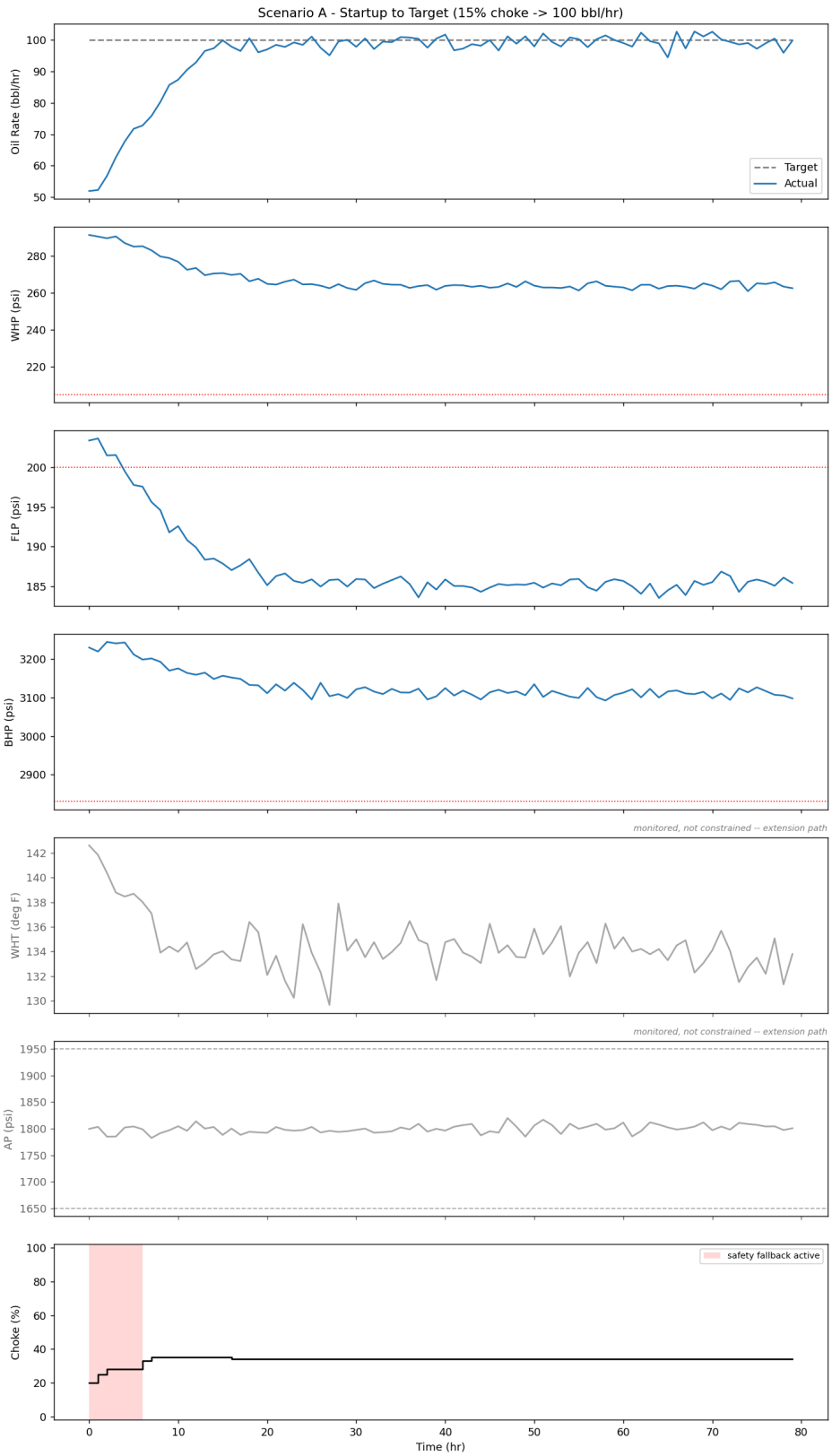
- Normal: *"Moved choke to 34.0% because it keeps WHP/FLP/BHP within safe limits over the next 12h and brings predicted oil rate to 99.4 bbl/hr, closest to the 100.0 bbl/hr target among 11 feasible options."*
- Safety fallback: *"No choke move keeps all limits satisfied over the lookahead; moved to 62.2% because it minimizes the predicted constraint violation (6.8 psi-steps over horizon)."*

Commanded choke values are rounded to 0.1% before being used for prediction, selection, *and* the logged string, so the rationale always quotes the exact value actually applied — it cannot silently diverge from what was commanded.

3. Results

All numbers below are from one representative run per scenario (default seeds); §3.5 reports the full 30-seed distribution, which is the number to trust over any single run.

3.1 Scenario A — Startup to Target (15% choke → 100 bbl/hr, 80h)

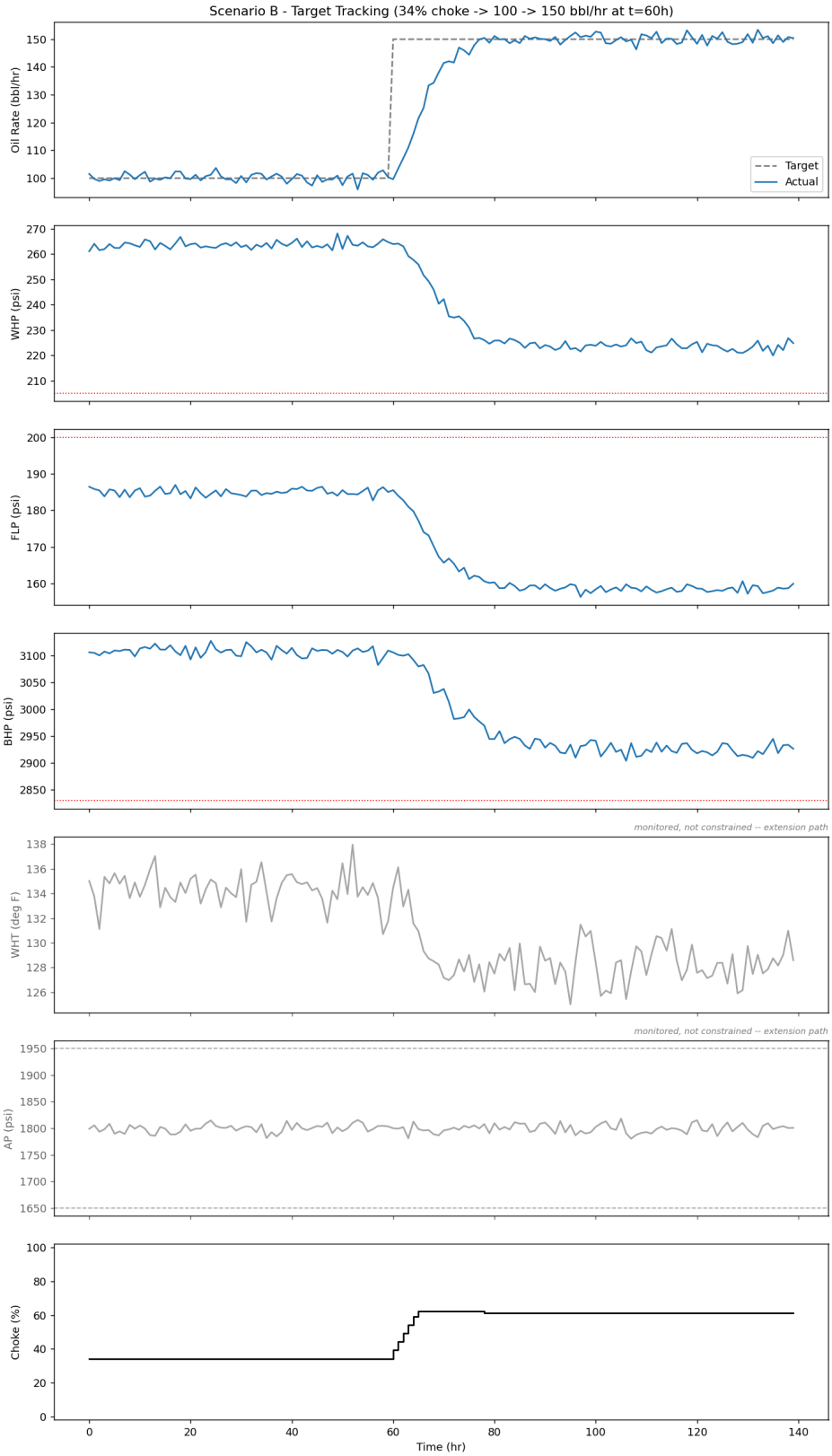


Scenario A: target vs. actual oil rate, WHP, FLP, BHP, and choke position over the 80h run

- **Tracking:** trailing 10-hour mean (hours 70–79) of **99.26 bbl/hr** at a steady 34.0% choke.
- **Safety, single-seed:** 4/80 constraint samples outside limits.
- **Safety, honest (30-seed sweep, see §3.5):** every single seed shows at least one violation (30/30), mean 3.97/80 steps, max 5/80.

- **Why, precisely (see §2.3):** this is not an "extrapolation artifact" — it's a deterministic consequence of the identified FLP steady-state curve exceeding the 200 psi ceiling below ~19% choke, combined with the ±5%/interval ramp limit. Scenario A starts at 15% (FLP initialized at its own steady state, 203.7 psi — already over) and the ramp limit caps how fast the choke can move toward a safe region, regardless of the controller's predictions.
- **New metric — time to enter the safe envelope:** the hour after which no further violations occur. Single-seed: **4h**. Across the 30-seed sweep: **mean 4.0h, range 3–5h** — a tight distribution driven almost entirely by the ramp-rate arithmetic (15%→20% in 1 step, →25% in 2, plus FLP's own τ=7h/θ=1h lag), not by noise. This tightness is itself evidence for the deterministic explanation: an extrapolation-uncertainty artifact would be expected to show more seed-to-seed spread than 3–5h.
- **Ramp-rate:** 0/80 violations in every seed — the ±5%/interval constraint was never breached; it's the *reason* the envelope takes several hours to reach, not itself a violated constraint.

3.2 Scenario B — Target Tracking (34.2% choke start, 100→150 bbl/hr step at t=60h, 140h)

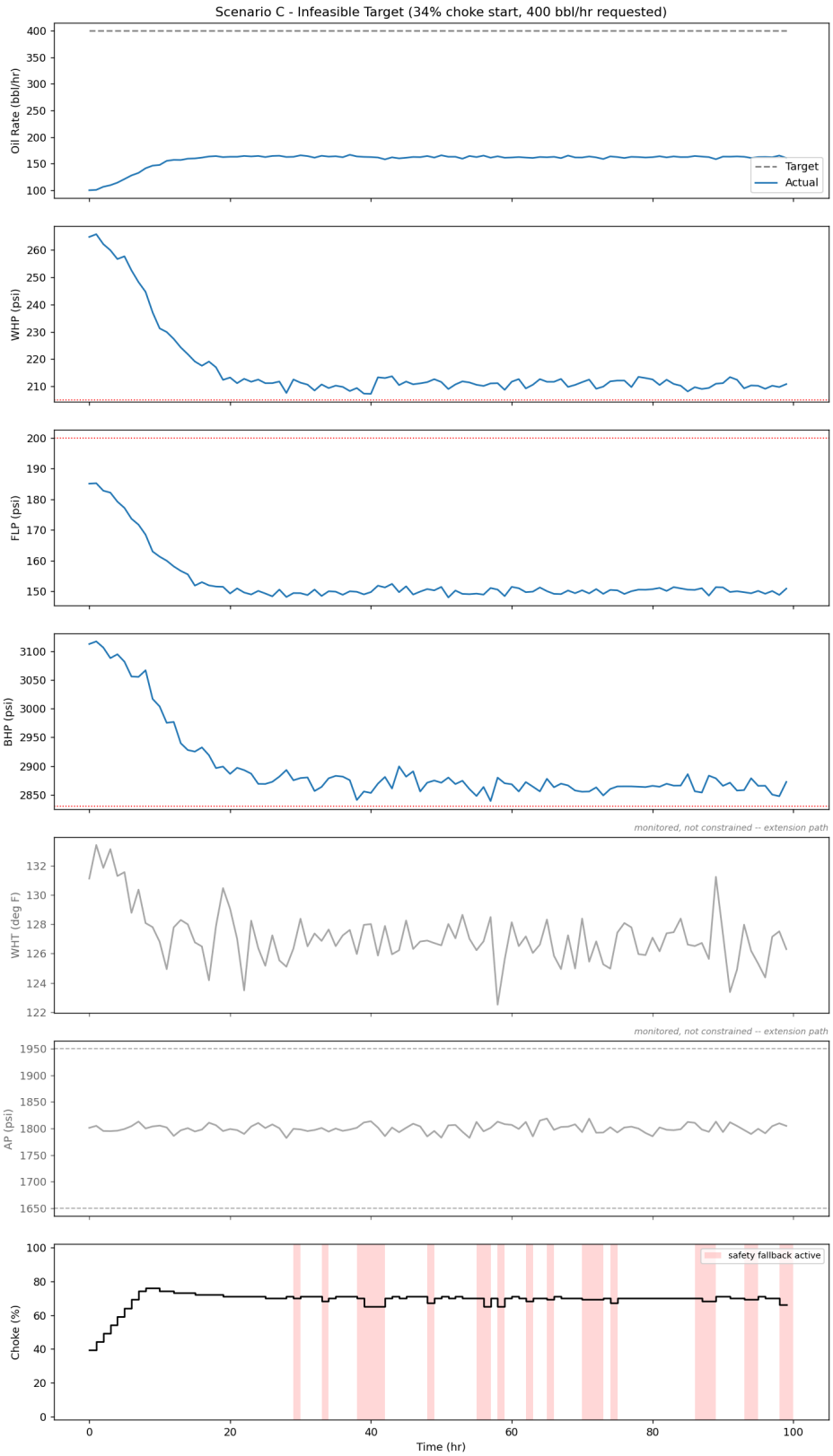


Scenario B: target vs. actual oil rate, WHP, FLP, BHP, and choke position over the 140h run, including the target step at t=60h

- **Tracking:** trailing 10-hour mean **100.32 bbl/hr** at 34.2% choke before the step (hours 50–59); **150.57 bbl/hr** at 61.2% choke after settling (hours 130–139).

- **Safety:** 0/140 in the representative run, and 0/140 in **all 30 seeds** — the only scenario with a perfectly clean sweep.

3.3 Scenario C — Infeasible Target (34.2% choke start, 400 bbl/hr requested, 100h)



Scenario C: target vs. actual oil rate, WHP, FLP, BHP, and choke position — the controller settles below the infeasible 400 bbl/hr target instead of chasing it into a violation

- **Behavior:** does not chase the infeasible target into a violation. Trailing 10-hour mean **162.76 bbl/hr** at 69.4% choke — the maximum rate the tightened envelope allows, not 400.
- **Safety, single-seed:** 0/100. **30-seed sweep: 0/100 in all 30 seeds** — fully clean.
- **Safety-fallback frequency:** 23.6% of steps (see §3.5) — far higher than A or B, because this scenario deliberately runs the choke near the edge of the tightened feasible envelope for its entire duration. Frequent fallback here is expected behavior, not a red flag: with 0 actual safety violations, the fallback branch is doing its job (picking the least-bad option under a flipping feasible set — see §3.4's root-cause discussion), not failing to.

3.4 Actuator activity: valve travel and move count

`MPCController.decide()` originally picked the feasible candidate purely by predicted target error (`q_err`), with move size only a last-resort tie-break. Once near target, many candidates have `q_err` differences smaller than measurement noise, so the "closest" one was effectively a coin flip decided by noise — the valve chattered chasing error it can't actually predict. Fixed by adding a move-suppression term to both branches' cost:

- Feasible branch: `cost = q_err + λ·|Δu|` (was: sort purely by `q_err`).
- Safety-fallback branch: `cost = violation + λ·|Δu|` (was: sort purely by `violation`) — added for the same reason; Scenario C spends 23.6% of its steps in this branch (§3.5), so it chatters just as easily if left unweighted.

`λ = 1.0` bbl/hr of predicted improvement required per %-point moved (a 1% move must be worth ≥1 bbl/hr; a 5% move must be worth ≥5 bbl/hr) — matching the target calibration exactly, not retuned to force a result.

Scenario	Moves	Total valve travel
A	5 / 80	16.0 %-pts
B	7 / 140	29.0 %-pts
C	53 / 100	129.0 %-pts

(C's travel was 154.0 %-pts before the move-suppression fix — the 129.0 above is post-fix, rendered fresh from `outputs/results.json` each time `generate_docs.py` runs, same as every other table in this document.)

Honest result, not fully positive: the fix works as intended for A and B — both settle and stay essentially still once near target (5 and 7 moves total, vs. constant hunting beforehand). **Scenario C is only partially improved**: total travel dropped 16% (154→129 %-pts) but move *count* is essentially

unchanged (52→53). Root cause, confirmed by inspecting the trajectory directly: C isn't chattering from a noise-driven tie between similarly-good candidates — it's riding the tightened WHP floor (208.6 psi, true limit 205 + 3σ margin) with WHP readings noisy enough (σ≈1.2 psi, observed swinging 207–213 psi) to repeatedly cross that threshold and flip which candidates are feasible at all. A move-suppression term on the *ranking* within whichever set is feasible that step can't fix a problem in *which set is feasible* changing step to step. A larger constraint-tightening margin (`noise_margin_sigma`, currently 3σ) would likely address this directly, but that's a different lever than the one asked for here and hasn't been tried.

3.5 Seed-sweep distribution (30 seeds per scenario, `seed_sweep.py`)

Scenario	Mean violations	Max violations	Seeds with ≥1 violation	Mean safety-fallback rate
A	3.97 / 80	5	30 / 30	8.58%
B	0.00 / 140	0	0 / 30	0.00%
C	0.00 / 100	0	0 / 30	23.57%

Full per-seed results: `outputs/seed_sweep_results.csv`.

3.6 Baseline comparison: MPC vs. Fixed-optimal vs. Fixed-operator-proxy vs. PI

Three baselines, run over identical scenarios/model/limits/seeds as the MPC for an apples-to-apples comparison:

- **Fixed-optimal** (`baselines.py`) — the choke the identified model says holds the target at steady state, walked back until its own steady-state predictions clear the tightened envelope, then held. Still model-informed and envelope-aware — this is what a good engineer *with* the identification pipeline would set, not a real operator without it. Models the "set once, left alone" half of pain point #2, but not pain point #1's conservatism.
- **Fixed-operator-proxy** (`baselines.py`) — no model, no envelope knowledge at all: a naive straight-line read of choke-vs-oil-rate off the raw reference CSV (the only data an operator without this pipeline would have), then backed off 15 percentage points from wherever that naive line says the target is met. Models pain point #1 directly: *"operators baby the choke conservatively (fear of sand/ formation damage), leaving real production capacity unused."*
- **PI** — velocity-form PI on oil rate, IMC-tuned from the identified model, blind to WHP/FLP/BHP by design.

Also added: **Scenario D — Disturbance Rejection** (`scenario_d.py`), a deliberate, explicit relaxation of the brief's "no changing reservoir properties" simplification, built specifically to test whether MPC's continuous re-planning beats a static setpoint once the plant genuinely drifts underneath both. BHP's identified steady-state offset drifts −0.5 psi/h for 200h (reservoir decline) at a constant 100 bbl/hr target;

the identified model itself is fit once, before the disturbance starts, and never updated — realistic to how a real deployment re-identifies occasionally, not continuously.

Scenario	Approach	Safety violations	Total barrels
A	MPC	4/80	7,590.4
A	Fixed-optimal	3/80	7,657.6
A	Fixed-operator-proxy	66/80	4,852.4
A	PI	3/80	7,652.1
B	MPC	0/140	17,676.4
B	Fixed-optimal	0/140	17,642.0
B	Fixed-operator-proxy	23/140	13,824.5
B	PI	0/140	17,591.0
C	MPC	0/100	15,806.2
C	Fixed-optimal	0/100	15,769.0
C	Fixed-operator-proxy	85/100	18,778.3
C	PI	85/100	18,778.3

Scenario D (`scenario_d.py`) isn't in `results.json` — a 200h run per approach is a different cost profile than the scripts above, not yet worth the wiring — so its row is hand-maintained here, re-verified against a fresh `python scenario_d.py` run before being typed in:

Scenario	Approach	Safety violations	Total barrels	Time-to-first-violation
D	MPC	0/200	20,040.9	never
D	Fixed-optimal	0/200	20,048.3	never
D	Fixed-operator-proxy	**121/200**	12,594.2	**21h**

The headline: what MPC actually buys over realistic manual operation. Fixed-operator-proxy stands in for a real operator working without this pipeline's model or envelope knowledge — the comparison that matters most, since it's the one this project is actually meant to replace (pain point #1). Barrel and per-day figures below are computed directly from `outputs/results.json` (barrel gain over each scenario's own run length, converted to a daily rate — not hand-typed):

- **Scenario A:** MPC produces **+2,738 bbl more** than the operator-proxy over the 80h run — **+821 bbl/day** — while cutting constraint violations from **66/80 down to 4/80**.
- **Scenario B: +3,852 bbl** over 140h — **+660 bbl/day** — while cutting violations from **23/140 to 0/140**.
- **Scenario C inverts, and that inversion is itself the result:** the operator-proxy nominally out-produces MPC by 2,972 bbl over the 100h run (**+713 bbl/day** in the proxy's favor) — but only by riding the choke wide open straight through the safety envelope, racking up **85/100** constraint violations against MPC's **0/100**. Output produced by blowing through a pressure limit isn't a result to chase; it's the exact failure mode this project exists to prevent.
- **Scenario D** (200h, hand-verified against a fresh `python scenario_d.py` run since it isn't in `results.json`) shows the same pattern at its cleanest: **+7,447 bbl** total, **+893 bbl/day**, **0/200 vs. 121/200** violations.

Scenario C's Fixed-operator-proxy and PI rows are identical (85/100 violations, 18,778.3 barrels) — checked, not a copy-paste artifact. Both approaches see the same simulator noise seed for Scenario C, and both independently drive the choke to its 100% hard ceiling and hold it there: the operator-proxy's setpoint function clips its believed setpoint to `CHOKE_MAX` (400 bbl/hr is far outside even its naive linear read), and PI's integrator saturates against the same ramp-clamped ceiling chasing an target it can never reach. Two different control laws converging to the identical fully-open trajectory, under the identical noise draw, produce identical readings — this is what a correct comparison looks like when a target is this infeasible, not a bug in either baseline.

A secondary, more surprising finding: MPC ties Fixed-optimal, its model-informed but non-adaptive sibling, in all four scenarios — including D, which was built specifically to find a gap between them. BHP and Q are independent FOPDT channels with no coupling in this model, so Scenario D's declining reservoir never moves Q off target and never drifts BHP close enough to its floor (ends ~170 psi above it after 200h) to matter — MPC's live re-planning had nothing to correct that Fixed-optimal's envelope-aware static setpoint didn't already handle. This is a real, checked result (both approaches hold choke within noise of 34.2% for the entire 200h run), not an assumption, but it's a narrower and less important claim than the headline above: the production and safety gains that actually matter come from envelope-awareness itself (present in both MPC and Fixed-optimal), not from live re-planning specifically.

The mechanism behind Fixed-operator-proxy's violations is more specific than "operators are conservative, therefore unsafe": backing off 15 points *closes* the choke, which *raises* WHP and BHP (both lower-bounded — this direction is safe) but also *raises* FLP (upper-bounded — this direction is unsafe). For a 100 bbl/hr target the naive belief-minus-15% lands at 18.7% choke, below the ~19% threshold (§2.3) where FLP's identified steady-state exceeds 200 psi. A real operator without envelope knowledge has no way to know that closing down is the *wrong* direction for that one constraint. Checked directly in Scenario D: violations start at hour 21 (as soon as the ramp-limited approach reaches 18.7%) and continue at a roughly steady rate for all 200 hours (17 in the first 50h vs. 33 in the last 50h) — the

reservoir decline is at most a minor secondary factor, not the primary cause. The same static-FLP-threshold mechanism that explains Scenario A's violations (§2.3) is what's actually failing here, not a failure to detect the disturbance. PI's 85/100 violations in Scenario C make the same point from a different angle: a controller with no predictive safety check — model-informed or not — will chase an infeasible target straight through the operating envelope.

Full results: `outputs/baseline_comparison.csv`, `outputs/scenario_D_mpc.csv`, `outputs/scenario_D_fixed_optimal.csv`, `outputs/scenario_D_fixed_operator_proxy.csv`.

3.7 Lessons learned

- **"Extrapolation" was a hedge, not the actual explanation — checking the model's own numbers found the real one.** Two consecutive earlier drafts got Scenario A's violations wrong: first claiming the 15% start was "inside supported territory" (false), then correctly retracting that but blaming the violations on generic "extrapolation uncertainty" (imprecise — it implies the numbers might not be real). Actually computing FLP's identified steady-state curve (§2.3) showed it exceeds the 200 psi ceiling below ~19% choke, full stop — a concrete, checkable fact about the current model, not a hedge about what an uncalibrated region *might* do. Combined with the ramp-rate limit, that fact alone guarantees several hours of violation from a 15% start, independent of whether the low-choke calibration is trustworthy. The 30-seed "time to enter the safe envelope" distribution (3–5h, mean 4.0h) is tight enough to support this as deterministic rather than noise-driven.
- **A correctly-specified model still shows real identification error — and the dwell-time lead was worth chasing down, not just naming.** §1.2's reframing — identify.py shares fitting code with the simulator by construction, so this isn't a black-box discovery exercise — turned what looked like a structural-mismatch problem into a testable hypothesis: finite dwell time relative to the true time constants. Raising `DWELL_HOURS` 24→40 confirmed it for three of four channels (WHP 16.0%→4.0%, FLP 27.1%→13.6%, BHP 31.1%→7.8%) and, just as usefully, *ruled it out* for Q (20.0%→19.0%, unchanged across 24–80h dwell tested) — leaving Q's bias correctly reported as a separate, still-open question rather than folded into an explanation that doesn't actually cover it. That open question turned out to have its own answer: a one-sample lag between the fitting code and the live simulator (§1.2), unrelated to dwell time, which dropped Q's error 19.0%→3.0% and made every channel's θ match exactly. Two separate root causes, found by refusing to let the first fix's partial success explain away the residual it didn't actually touch.
- **Name the controller accurately.** Calling this "brute-force MPC" implied guarantees (recursive feasibility, trajectory optimality) it doesn't have. It's a one-step receding-horizon search with hold-constant prediction, and its safety property is empirically strong (§3.5) but not formally proven. Both things can be true, and only naming it precisely lets a reader hold both.
- **Report distributions, not single runs.** A single seed's "0 violations" or "104.0 bbl/hr" is a point sample from a noisy system. The 30-seed sweep is what actually supports a safety claim (Scenario B

is clean in all 30 seeds; Scenario A violates in all 30) or a tracking claim (trailing 10-hour means, not one timestamp).

- **A correction layer earns its place by generalizing, or it doesn't ship — channel by channel, not as a blanket decision.** §1.3's correction helps three of four channels (Q, FLP, BHP) after the timing-lag fix, and is skipped only for WHP. Both outcomes come from the same held-out-RMSE rule with no manual override — the discipline is the point, not any specific channel's verdict, which is also why this list changed (from "BHP only" to "three of four") without anyone hand-editing which channels are marked used.